Name: T.MOHANA

Hall Ticket: 2303A52446

Batch: 41

Task 1: AI-Assisted Pair Programming

Prompt:

Generate a basic Python expense tracker application that records daily expenses with categories and then improve it by adding features like monthly expense summary, proper input validation, and refactoring the code into functions for better structure and maintainability

Code:

```
class ExpenseTracker:
    def __init__(self):
        self.expenses = []

    def add_expense(self, amount, category):
        if not self.validate_amount(amount):
            print("Invalid amount. Please enter a positive number.")
            return
        if not self.validate_category(category):
            print("Invalid category. Please enter a non-empty string.")
            return
        self.expenses.append({'amount': amount, 'category': category})
        print("Expense added successfully.")

    def validate_amount(self, amount):
        try:
            amount = float(amount)
            return amount > 0
        except ValueError:
            return False

    def validate_category(self, category):
        return isinstance(category, str) and category.strip() != ""

    def monthly_summary(self):
        summary = {}
        for expense in self.expenses:
            category = expense['category']
            amount = expense['amount']
            if category in summary:
                summary[category] += amount
            else:
                summary[category] = amount
        return summary
def main():
    tracker = ExpenseTracker()
    while True:
        amount = input("Enter expense amount (or 'exit' to finish): ")
        if amount.lower() == 'exit':
            break
```

```
        category = input("Enter expense category: ")
        tracker.add_expense(amount, category)

    print("\nMonthly Expense Summary:")
    summary = tracker.monthly_summary()
    for category, total in summary.items():
        print(f"{category}: ${total:.2f}")
if __name__ == "__main__":    main()
```

Output:

```
PS C:\Users\lilac\New folder\fashion-store\server> &
erver/ai5.py"
Enter expense amount (or 'exit' to finish): 390
Enter expense category: dress
Expense added successfully.
Enter expense amount (or 'exit' to finish):
```

Explanation:

This code first creates a simple expense tracker and then improves it by adding better structure and useful features. It shows how AI helps in refining code through collaboration and enhancements.

Task 2: **Version Control Integration**

Prompt:

Create a Python student grade calculator project and demonstrate version control using Git by initializing a repository, creating a feature branch to add grade classification and average/percentage calculations, committing changes with meaningful messages, and merging the branch back into the main branch with proper conflict handling

Code:

```
import re
def calculate_grade(marks):
    if marks >= 90:
        return 'A'
    elif marks >= 80:
        return 'B'
    elif marks >= 70:
        return 'C'
    elif marks >= 60:
        return 'D'
    else:
        return 'F'
def main():
    student_name = input("Enter the student's name: ")
    marks_input = input("Enter the marks (0-100): ")

    if not re.match(r'^\d{1,3}$', marks_input):
        print("Invalid input. Please enter a number between 0 and 100.")
        return

    marks = int(marks_input)
```

```
    if marks < 0 or marks > 100:
        print("Marks should be between 0 and 100.")
        return

    grade = calculate_grade(marks)
    percentage = (marks / 100) * 100

    print(f"Student Name: {student_name}")
    print(f"Marks: {marks}")
    print(f"Grade: {grade}")
    print(f"Percentage: {percentage:.2f}%")
if __name__ == "__main__":
    main()
```

Bash:

git init

git add .

git commit -m "Initial grade calculator"

git branch feature-ai

git checkout feature-ai

git add .

git commit -m "Added grade classification and percentage"

git checkout main

git merge feature-ai

Output:

```
PS C:\Users\lilac\New folder\fashion-store\server> git commit -m "Grade Calculator"
 create mode 100644 test.http.txt
 create mode 100644 uploads/Ai-Assisted-Coding.pdf
 create mode 100644 users.db
PS C:\Users\lilac\New folder\fashion-store\server> git branch feature-ai
PS C:\Users\lilac\New folder\fashion-store\server> git checkout feature-ai
Switched to branch 'feature-ai'
PS C:\Users\lilac\New folder\fashion-store\server> git add .
PS C:\Users\lilac\New folder\fashion-store\server> git commit -m "Added grade classification and perce
On branch feature-ai
nothing to commit, working tree clean
PS C:\Users\lilac\New folder\fashion-store\server> git checkout main
error: pathspec 'main' did not match any file(s) known to git
PS C:\Users\lilac\New folder\fashion-store\server> git merge feature-ai
Already up to date.
```

Explanation:

This task shows how Git is used to manage and track changes in a project while adding new features. It demonstrates collaborative coding by using branches, commits, and merging improvements safely.

Task 3: AI for Commit Message Generation

Prompt:

Generate a Python password validation module and then modify it by adding rules like minimum length and special character requirements, and use AI to generate clear and meaningful Git commit messages describing these changes

Code :

```python
# Password validation module
import re

def validate_password(password):
    # Check minimum length
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."

    # Check for special characters
    if not re.search(r'[!@#$%^&*()-+]', password):
        return False, "Password must contain at least one special character."

    # Check for uppercase letters
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least one uppercase letter."

    # Check for lowercase letters
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter."

    # Check for digits
    if not re.search(r'\d', password):
        return False, "Password must contain at least one digit."

    return True, "Password is valid."
# Example usage
password = input("Enter a password: ")
is_valid, message = validate_password(password)
print(message)
# Git commit message: "Added password validation rules for minimum length, special
characters, uppercase letters, lowercase letters, and digits."
```

Output:

```
PS C:\Users\lilac\New folder\fashion-store\server> & C:
erver/ai5.py"
Enter a password: L!k!th@123
Password is valid.
PS C:\Users\lilac\New folder\fashion-store\server>
```

Explanation:

This task shows how AI can help generate clear and structured commit messages for code changes. It also compares AI-generated messages with human-written ones to evaluate clarity and usefulness.

Task 4: Pair Programming Simulation

Prompt:

Generate a basic Python quiz management system with questions and answers and then improve it by adding features like score calculation and input validation, simulating pair programming where one version is initial and another enhanced version is merged using Git branching

Code:

```
# Initial version of the quiz management system
class Quiz:
    def __init__(self):
        self.questions = []
        self.score = 0

    def add_question(self, question, answer):
        self.questions.append((question, answer))

    def take_quiz(self):
        for question, answer in self.questions:
            user_answer = input(question + " ")
            if user_answer.lower() == answer.lower():
                print("Correct!")
                self.score += 1
            else:
                print("Wrong! The correct answer is:", answer)

    def show_score(self):
        print(f"Your score is: {self.score}/{len(self.questions)}")
# Enhanced version with input validation and improved score calculation
class Quiz:
    def __init__(self):
        self.questions = []
        self.score = 0

    def add_question(self, question, answer):
        self.questions.append((question, answer))

    def take_quiz(self):
        for question, answer in self.questions:
            while True:
                user_answer = input(question + " ")
                if user_answer.strip() == "":
                    print("Input cannot be empty. Please try again.")
                else:
                    break
            if user_answer.lower() == answer.lower():
                print("Correct!")
                self.score += 1
            else:
                print("Wrong! The correct answer is:", answer)

    def show_score(self):
        print(f"Your score is: {self.score}/{len(self.questions)}")
```

```python
# Example usage
quiz = Quiz()
quiz.add_question("What is the capital of France?", "Paris")
quiz.add_question("What is 2 + 2?", "4")
quiz.take_quiz()
quiz.show_score()
```

Bash:

git init

git add .

git commit -m "Initial quiz system with questions"

git branch feature-enhancement

git checkout feature-enhancement

git add .

git commit -m "Added score calculation and input validation"

git checkout main

git merge feature-enhancement

Output:

```
What is the capital of France? Paris
Correct!
What is 2 + 2? 4
Correct!
Your score is: 2/2
PS C:\Users\lilac\New folder\fashion-store\server> git init
Reinitialized existing Git repository in C:/Users/lilac/New folder/fashion-store/server/.git/
PS C:\Users\lilac\New folder\fashion-store\server> git add .
PS C:\Users\lilac\New folder\fashion-store\server> git commit -m "Initial quiz system with questions"
[feature-ai 3d36364] Initial quiz system with questions
 1 file changed, 52 insertions(+), 36 deletions(-)
PS C:\Users\lilac\New folder\fashion-store\server> git branch feature-enhancement
PS C:\Users\lilac\New folder\fashion-store\server> git checkout feature-enhancement
Switched to branch 'feature-enhancement'
PS C:\Users\lilac\New folder\fashion-store\server> git add .
PS C:\Users\lilac\New folder\fashion-store\server> git commit -m "Added score calculation and input validation"
On branch feature-enhancement
nothing to commit, working tree clean
PS C:\Users\lilac\New folder\fashion-store\server> git checkout main
error: pathspec 'main' did not match any file(s) known to git
PS C:\Users\lilac\New folder\fashion-store\server> git merge feature-enhancement
Already up to date.
PS C:\Users\lilac\New folder\fashion-store\server>
```

Explanation:

This task demonstrates how two developers collaborate by improving the same code using AI assistance. It shows how features are added step by step and integrated using version control.